Lab 2 Scanning Networks

> **Disclaimer : Using the notes is for educational purpose only, the owner is not responsible for any misconduct by the reader**

Lab 2-01 Perform Host Discovery using Nmap

Scenario:

A mid-sized IT company, TechSec Solutions, is enhancing its cybersecurity posture and has tasked its internal security team to assess its network thoroughly. The team must identify open ports and running services across the organization's network as part of this assessment. This discovery will help them detect potential vulnerabilities and secure their systems proactively. The network infrastructure includes a mix of Linux and Windows servers, database systems, and IoT devices operating within a predefined subnet range. Additionally, the company uses IPv6-enabled devices to support their next-generation networking needs.

Solution:

Phase 1: Understanding the document and scope

Before proceeding with vulnerability assessment and open port enumeration, it is essential to clearly understand the organization's primary requirement. The objective is to ensure that all services and connected devices are secure and not exposed to potential vulnerabilities.

As part of the penetration testing engagement, the first step is to identify the total number of live hosts within the defined scope. Accurate host discovery ensures that no active system is overlooked during subsequent assessment phases.

Lab Environment Overview

For this scenario, the testing environment consists of:

- **Attacker Machine:** Kali Linux
- **Target Machine:** Metasploitable (192.168.1.64)

All systems are connected within the same network using a bridged adapter configuration on a personal test network. This setup simulates a realistic internal network environment for controlled security testing.

Host Discovery Approach

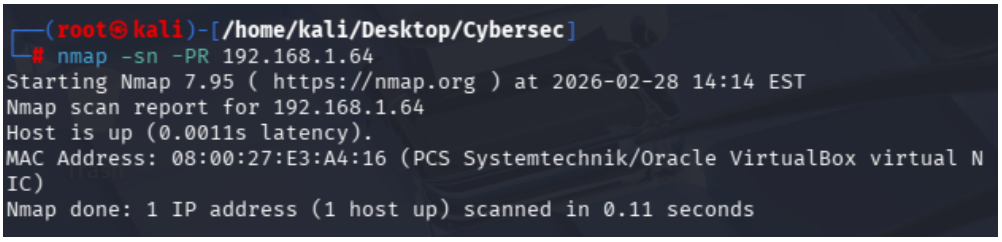
To enumerate active systems within the network range, multiple tools are available, such as `netdiscover`, `bettercap`, and `nmap`. For this phase, `nmap` is selected as the primary tool due to its reliability and flexibility.

Phase 2: Understanding Different Types of Nmap Host Discovery Scans

Phase 2.1: ARP Ping Scan

```
nmap -sn -PR 192.168.1.64
```

Explanation: An ARP Ping Scan identifies live hosts by sending ARP request broadcasts within the local subnet. Active systems respond with ARP replies containing their MAC addresses, confirming availability without performing any port scanning. The `-sn` flag disables port scanning, and `-PR` specifies ARP-based discovery.

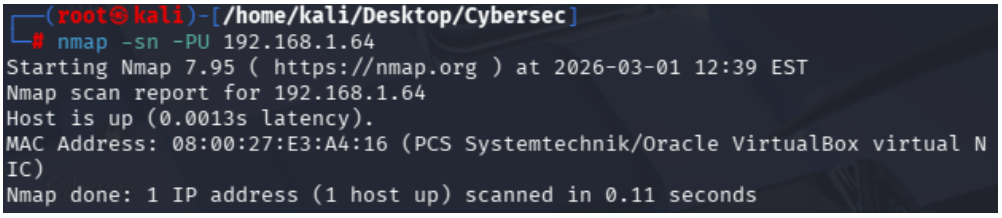


Exception / When to use this: This method is executed **only** when the attacker and target are on the **same Layer 2 broadcast domain (local network segment)**. It does not function across routers or different subnets. On Metasploitable, since both machines are on the same LAN, this is the fastest and most reliable host discovery method — it bypasses all IP-layer firewalls because ARP operates at Layer 2.

Phase 2.2: UDP Ping Scan

```
nmap -sn -PU 192.168.1.64
```

Explanation: Performs host discovery by sending UDP packets to the target. If the host is alive, it typically responds with an ICMP "Port Unreachable" message, confirming the system is online even though the UDP port is closed.

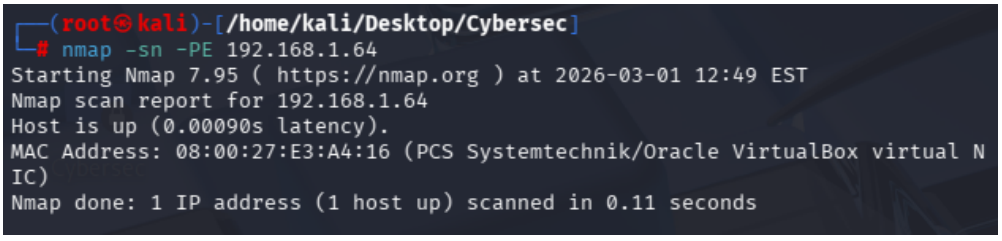


Exception / When to use this: This scan is used **only** when ICMP Echo Requests (ping) are blocked by the target or an intermediate firewall, but UDP responses are still permitted. On Metasploitable, this is useful in scenarios where a firewall rule drops ICMP but allows ICMP error responses back. It will fail if the firewall silently drops both UDP probes and ICMP error messages.

Phase 2.3: ICMP Echo Ping Scan

```
nmap -sn -PE 192.168.1.64
```

Explanation: Sends ICMP Echo Request (Type 8) packets to the target. If the system is active and ICMP is permitted, it responds with an ICMP Echo Reply (Type 0), confirming host availability. Classic "ping" behavior, structured into nmap's workflow.



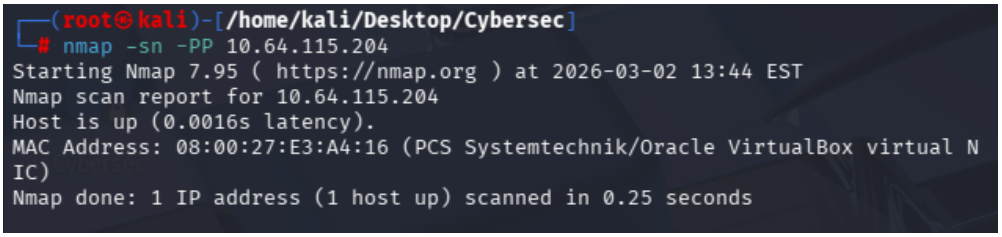
Exception / When to use this: This scan is executed **only** when ICMP traffic is permitted through the network and firewall. It is the simplest, most universal check and is used as a baseline. It will fail if the target OS or firewall has ICMP echo requests filtered — common on Windows systems with Windows Firewall enabled or hardened Linux hosts.

Know more about types? Checkout here

Phase 2.4: ICMP Timestamp Ping Scan

```
nmap -sn -PP 10.64.115.204
```

Explanation: Sends ICMP Timestamp Request packets (Type 13) instead of standard echo requests. If the host is alive, it responds with a Timestamp Reply (Type 14), revealing that the host is up even if echo requests are blocked.

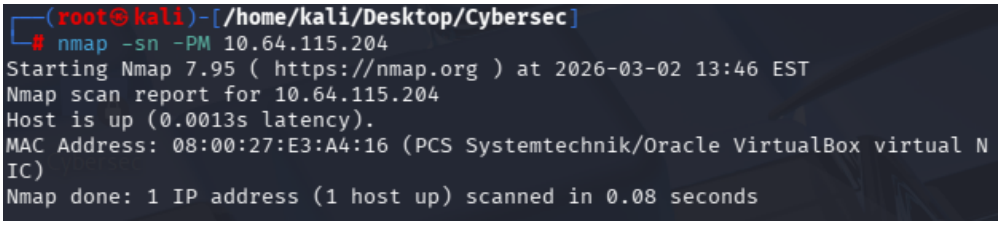


Exception / When to use this: This is used **only** when ICMP Echo (Type 8) is explicitly blocked by a firewall rule but Type 13 Timestamp messages are not — a common misconfiguration in older or improperly configured firewall rulesets. On Metasploitable, this helps confirm host availability when `-PE` yields no response.

Phase 2.5: ICMP Address Mask Ping Scan

```
nmap -sn -PM 10.64.115.204
```

Explanation: Sends ICMP Address Mask Request packets (Type 17) to the target. A live host may respond with an Address Mask Reply (Type 18). This is a legacy ICMP mechanism originally used to determine the subnet mask.

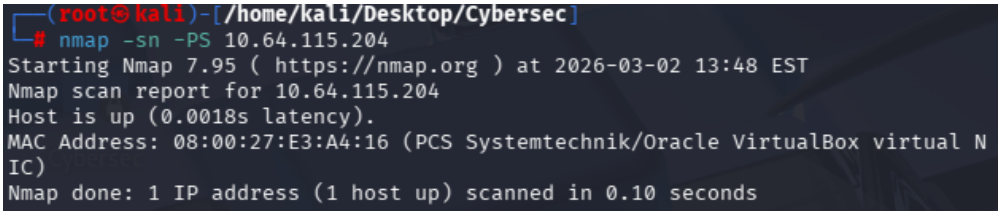


Exception / When to use this: This technique is used **only** when both ICMP Echo and Timestamp are filtered but the firewall does not explicitly block Type 17/18 ICMP messages. It is rarely reliable on modern systems but can work against older or legacy targets like Metasploitable where iptables rules are minimal.

Phase 2.6: TCP SYN Ping Scan

```
nmap -sn -PS 10.64.115.204
```

Explanation: Sends a TCP SYN packet to the target (default port 80). If the host is alive, it responds with SYN/ACK (port open) or RST (port closed) — both confirm the host is live. Nmap then sends RST to close the connection without completing the handshake.

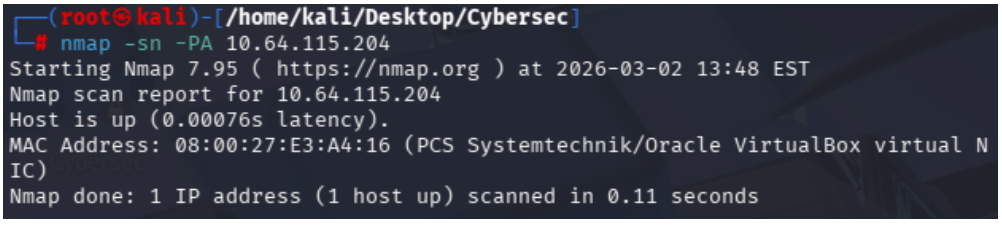


Exception / When to use this: This scan is executed **only** when ICMP-based discovery is entirely blocked and the target has at least one TCP port reachable. It is the preferred host discovery method against internet-facing hosts where ICMP is firewalled. On Metasploitable (10.64.115.204), this is highly effective since it runs many open TCP services (port 80, 22, 21, etc.).

Phase 2.7: TCP ACK Ping Scan

```
nmap -sn -PA 10.64.115.204
```

Explanation: Sends a TCP ACK packet to the target (default port 80). A stateless firewall may let ACK packets through since they look like part of an established connection. A live host responds with RST, confirming it is alive.



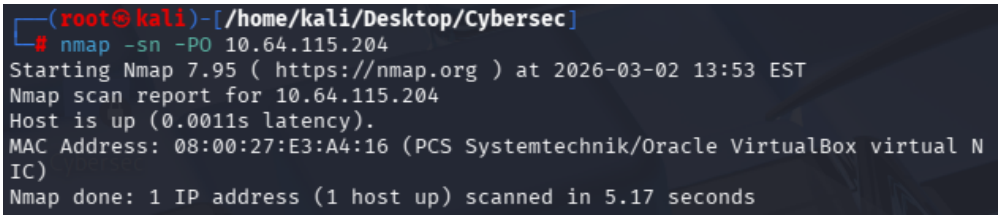
Exception / When to use this: This is used **only** when the network uses **stateless packet filtering** (older ACL-based firewalls) that allow ACK packets through but block SYN packets. Stateful firewalls will block ACK probes to ports without an

existing connection. It complements `-PS` in environments where SYN packets are filtered but ACK packets are not.

Phase 2.8: IP Protocol Ping Scan

```
nmap -sn -PO 10.64.115.204
```

Explanation: Sends multiple IP packets using different protocol numbers (ICMP, IGMP, IP-in-IP by default) to determine if the host is live. Any response from the target — whether an error or a protocol reply — confirms host availability.

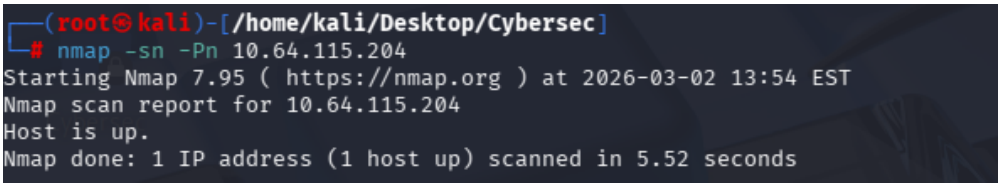


Exception / When to use this: This scan is used **only** when conventional ICMP and TCP-based discovery methods are all blocked, but certain IP-layer protocols may be allowed. Useful against niche or embedded network devices on Metasploitable that respond to unusual IP protocols. It is a last-resort host discovery technique when all other ping types fail.

Phase 2.9: No Ping / Treat All Hosts as Online

```
nmap -Pn 10.64.115.204
```

Explanation: Skips host discovery entirely and assumes the target is online, proceeding directly to port scanning. Every IP specified will be scanned regardless of ping responses.



Exception / When to use this: This is used **only** when the target is confirmed to be online but all ping-based discovery methods are blocked by a firewall, making nmap falsely conclude the host is down. This is the standard flag used against heavily

firewalled hosts, cloud instances, or when you already know from prior intelligence that 10.64.115.204 (Metasploitable) is live.

Lab 2-02 Perform Port and Service Discovery using Nmap

Scenario:

Following host discovery, the security team at TechSec Solutions must now enumerate open ports and identify the services running on each discovered host. The objective is to map the attack surface of Metasploitable (10.64.115.204) by identifying exploitable services, misconfigured daemons, and their exact version numbers to support subsequent vulnerability assessment.

Solution:

Phase 1: Document Scope and Understanding

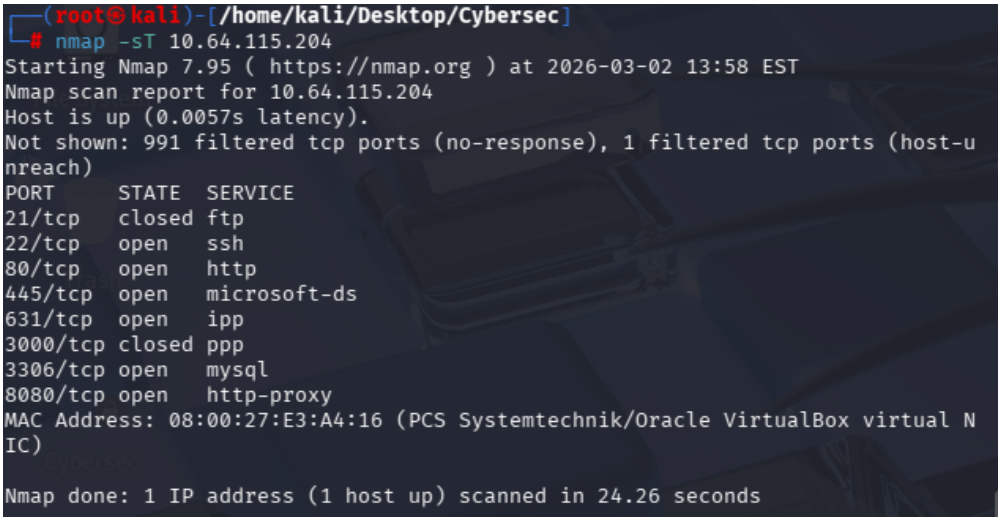
Port scanning is the systematic process of probing a host's TCP and UDP ports to determine their state — open, closed, or filtered — and identify the services listening on them. Each scan technique has unique characteristics in terms of stealth, reliability, and the information it yields.

Port State	Meaning	Open	A service is actively listening on this port and accepting connections.
Closed	The port is accessible but no service is listening. The host responds with RST.	**Filtered**	A firewall is blocking probes; nmap cannot determine if the port is open or closed.
Unfiltered	Port is accessible but nmap cannot determine open/closed (used in ACK scans).	**Open|Filtered**	Nmap cannot determine whether the port is open or filtered (common in UDP scans).

Phase 2: TCP Connect Scan (Full Open Scan)

```
nmap -sT 10.64.115.204
```

Explanation: Performs a full TCP three-way handshake (SYN → SYN/ACK → ACK) with each target port. If the connection succeeds, the port is open; if RST is received, the port is closed. Nmap then sends FIN to gracefully close the connection. This is the most reliable scan as it uses the OS's own network stack.



```
┌──(root㉿kali)-[/home/kali/Desktop/Cybersec]
└─# nmap -sT 10.64.115.204
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-02 13:58 EST
Nmap scan report for 10.64.115.204
Host is up (0.0057s latency).
Not shown: 991 filtered tcp ports (no-response), 1 filtered tcp ports (host-u
nreach)
PORT     STATE  SERVICE
21/tcp   closed ftp
22/tcp   open   ssh
80/tcp   open   http
445/tcp  open   microsoft-ds
631/tcp  open   ipp
3000/tcp closed ppp
3306/tcp open   mysql
8080/tcp open   http-proxy
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 24.26 seconds
```

Exception / When to use this: This scan is executed **only** when the attacker does **not** have raw socket privileges (i.e., running nmap without root/sudo) or when reliability is more important than stealth. It is noisier than SYN scans because the full connection is logged by most services and firewalls. On Metasploitable, this guarantees accurate results even under non-privileged user conditions.

Phase 2.1: TCP SYN Scan (Half-Open / Stealth Scan)

```
nmap -sS 192.168.1.65
```

Explanation: Sends a SYN packet; if SYN/ACK is received the port is open, and nmap immediately sends RST without completing the handshake. Also called a "half-open" scan. Since the connection is never fully established, it bypasses basic logging in many applications.

```
┌──(root㉿kali)-[/home/kali/Desktop/Cybersec]
└─# nmap -sS 192.168.1.65
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-02 14:13 EST
Nmap scan report for 192.168.1.65
Host is up (0.00086s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   closed ftp
22/tcp   open   ssh
80/tcp   open   http
445/tcp  open   microsoft-ds
631/tcp  open   ipp
8000/tcp closed ppp
3306/tcp open   mysql
8080/tcp open   http-proxy
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 10.26 seconds
```

Exception / When to use this: This is the **default scan** when nmap is run with root privileges. It is used when stealth is required — many older intrusion detection systems and application-level logging mechanisms only record fully established connections. It requires raw socket access (root). On Metasploitable, this is the standard first scan to enumerate all open TCP ports quickly and with minimal noise.

Phase 2.2: UDP Scan

```
nmap -sU 192.168.1.65
```

Explanation: Sends UDP packets to target ports. An ICMP "Port Unreachable" response means the port is closed. No response means the port is open or filtered (services like DNS, SNMP, DHCP use UDP). This scan is inherently slower because the OS rate-limits ICMP error responses.

```
┌──(root㉿kali)-[/home/kali/Desktop/Cybersec]
└─# nmap -sU 192.168.1.65
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-02 14:16 EST
Nmap scan report for 192.168.1.65
Host is up (0.00094s latency).
All 1000 scanned ports on 192.168.1.65 are in ignored states.
Not shown: 1000 open|filtered udp ports (no-response)
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 27.05 seconds
```

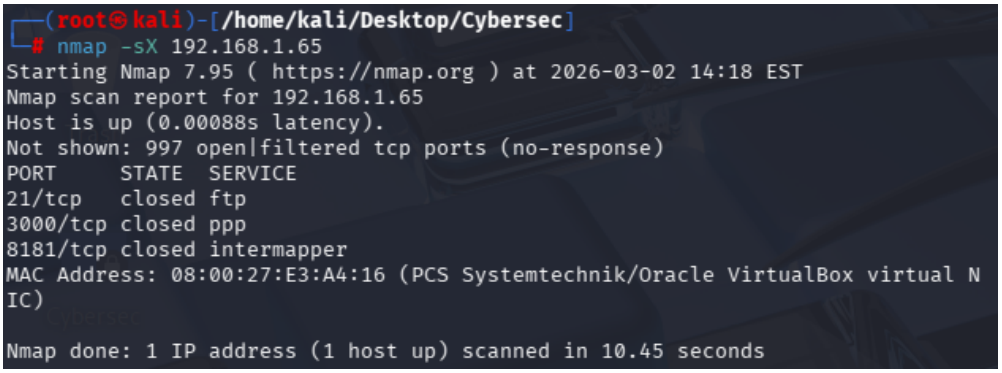
Exception / When to use this: This scan is used **only** when TCP scanning is insufficient and discovery of UDP-based services is required. On Metasploitable, critical services like SNMP (port 161) and TFTP run over UDP and would be missed

by TCP-only scans. This scan is computationally expensive and slow — it is run only against specific targets and often combined with `--top-ports` to limit scope.

Phase 2.3: XMAS Scan

```
nmap -sX 192.168.1.65
```

Explanation: Sends packets with FIN, PSH, and URG flags set simultaneously (all "lit up" like a Christmas tree). On RFC-compliant systems, open ports ignore this packet while closed ports respond with RST. Firewalled ports produce no response.



```
┌──(root㉿kali)-[/home/kali/Desktop/Cybersec]
└─# nmap -sX 192.168.1.65
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-02 14:18 EST
Nmap scan report for 192.168.1.65
Host is up (0.00088s latency).
Not shown: 997 open|filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   closed ftp
3000/tcp closed ppp
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 10.45 seconds
```

Exception / When to use this: This scan is executed **only** against **UNIX/Linux-based targets** when stealth is critical, as it does not appear in TCP connection logs. It will **not work** against Windows systems (which respond with RST regardless of port state, violating RFC 793). On Metasploitable (a Linux system), this scan effectively distinguishes open from closed ports without completing any TCP connection.

Phase 2.4: NULL Scan

```
nmap -sN 192.168.1.65
```

Explanation: Sends TCP packets with no flags set (null packet). RFC-compliant systems respond with RST if the port is closed and remain silent if the port is open. This makes open ports invisible to many basic firewall rules that look for standard flags.

```
┌──(root㉿kali)-[/home/kali/Desktop/Cybersec]
└─# nmap -sN 192.168.1.65
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-02 14:19 EST
Nmap scan report for 192.168.1.65
Host is up (0.00086s latency).
Not shown: 997 open|filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   closed ftp
3000/tcp closed ppp
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 9.99 seconds
```

Exception / When to use this: This scan is executed **only** against **RFC 793-compliant Unix/Linux systems** for maximum stealth. It bypasses packet filters that only monitor for SYN packets. Like the XMAS scan, it **fails against Windows** hosts. On Metasploitable, this is used when trying to stay below IDS detection thresholds by using anomalous, flagless packets.

Phase 2.5: FIN Scan

```
nmap -sF 192.168.29.163
```

Explanation: Sends a TCP FIN packet to each port. Closed ports respond with RST; open ports silently ignore the packet per RFC 793. This exploits the TCP specification to infer port states without initiating any connection.

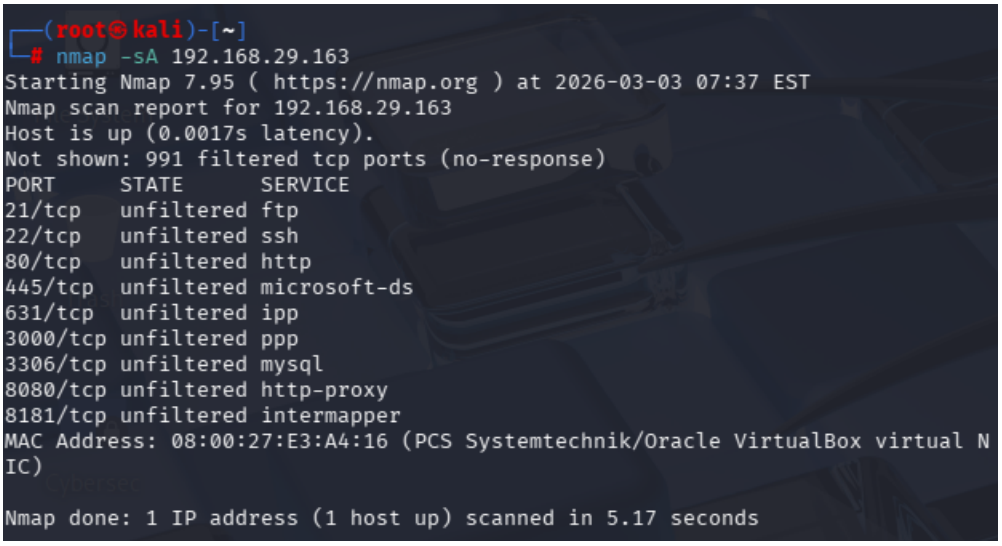
```
┌──(root㉿kali)-[~]
└─# nmap -sF 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 06:56 EST
Nmap scan report for 192.168.29.163
Host is up (0.00092s latency).
Not shown: 998 open|filtered tcp ports (no-response)
PORT     STATE  SERVICE
3000/tcp closed ppp
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 8.36 seconds
```

Exception / When to use this: This scan is executed **only** when a full SYN or connect scan would trigger IDS alerts and the target is a **Linux/Unix-based** system. FIN packets are less likely to be flagged by stateless packet filters that primarily inspect for SYN packets. On Metasploitable, this is a solid stealth alternative to `-sS` when basic stateless firewalling is in place.

Phase 2.6: ACK Scan

```
nmap -sA 192.168.29.163
```

Explanation: Sends TCP ACK packets to determine firewall rulesets, not to find open ports. An "unfiltered" result (RST received) means the packet reached the host. A "filtered" result (no response or ICMP error) indicates a firewall is in the way.



```
┌──(root㉿kali)-[~]
└─# nmap -sA 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 07:37 EST
Nmap scan report for 192.168.29.163
Host is up (0.0017s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE      SERVICE
21/tcp   unfiltered ftp
22/tcp   unfiltered ssh
80/tcp   unfiltered http
445/tcp  unfiltered microsoft-ds
631/tcp  unfiltered ipp
3000/tcp unfiltered ppp
3306/tcp unfiltered mysql
8080/tcp unfiltered http-proxy
8181/tcp unfiltered intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 5.17 seconds
```

Exception / When to use this: This scan is executed **only** to **map firewall rules** and determine which ports are filtered vs. unfiltered — **not** to find open services. It is used after initial port discovery when understanding the firewall architecture is needed. On Metasploitable, which has minimal iptables rules, most ports show as unfiltered, confirming the absence of strict egress/ingress filtering.

Phase 2.7: Window Scan

```
nmap -sW 192.168.29.163
```

Explanation: Similar to ACK scan but examines the TCP Window field in RST responses. Some systems set a non-zero window value for open ports and zero for closed ports in their RST reply, allowing differentiation between open and closed ports.

```
┌──(root㉿kali)-[~]
└─# nmap -sW 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 07:38 EST
Nmap scan report for 192.168.29.163
Host is up (0.0020s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   closed ftp
22/tcp   closed ssh
80/tcp   closed http
445/tcp  closed microsoft-ds
631/tcp  closed ipp
3000/tcp closed ppp
3306/tcp closed mysql
8080/tcp closed http-proxy
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 5.70 seconds
```

Exception / When to use this: This scan is executed **only** when targeting specific OS implementations that exhibit this window size behavior in RST packets — typically **BSD-based systems and certain older Linux kernels**. It is not universally reliable. On Metasploitable, it may yield inconsistent results but is used as a supplementary technique when `-sA` returns only "unfiltered" states without distinguishing open from closed.

Phase 2.8: Maimon Scan

```
nmap -sM 192.168.29.163
```

Explanation: Sends FIN/ACK packets. On some BSD-derived systems, open ports drop the packet while closed ports send RST. Named after Uriel Maimon who discovered this behavior in 1996.

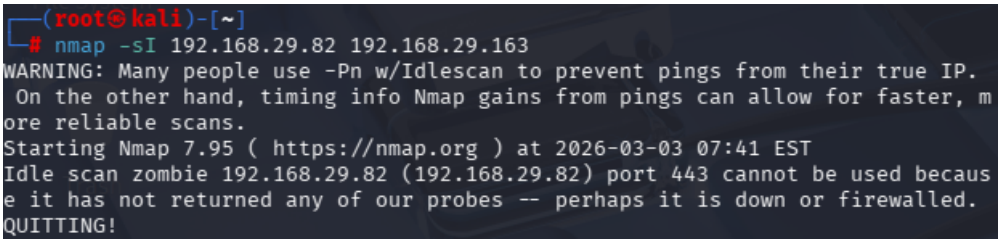
```
┌──(root㉿kali)-[~]
└─# nmap -sM 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 07:39 EST
Nmap scan report for 192.168.29.163
Host is up (0.0014s latency).
Not shown: 991 open|filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   closed ftp
22/tcp   closed ssh
80/tcp   closed http
445/tcp  closed microsoft-ds
631/tcp  closed ipp
3000/tcp closed ppp
3306/tcp closed mysql
8080/tcp closed http-proxy
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 5.84 seconds
```

Exception / When to use this: This scan is executed **only** when targeting **BSD-derived Unix systems** where this specific TCP stack behavior has been confirmed. It is largely a niche historical technique. On Metasploitable (Debian-based Linux), this scan will typically produce RST for all ports since modern Linux does not exhibit the Maimon behavior, making it useful as a negative control test.

Phase 2.9: Idle / Zombie Scan

```
nmap -sI <zombie_ip> 192.168.29.163
```

Explanation: The most covert scan technique. Nmap uses an "idle" third-party host (zombie) as a proxy. It monitors the zombie's IPID sequence to infer port states on the target — the attacker's IP never directly touches the target, making it nearly unattributable.



Exception / When to use this: This scan is executed **only** when **complete anonymity** is required and a suitable idle zombie host (one with predictable, incrementing IPID values) is available on the network. It is used in advanced red team engagements where attribution must be avoided. On a controlled Metasploitable lab network, this demonstrates the concept of indirect scanning — replace `<zombie_ip>` with any idle host on the 192.168.1.0/24 subnet.

Phase 2.10: Service Version Detection

```
nmap -sV 192.168.29.163
```

Explanation: After identifying open ports, nmap probes each open port with service-specific payloads (banners, protocol handshakes) to determine the exact service name, version number, and sometimes the underlying application. This data directly feeds into vulnerability matching.

```
┌──(root㉿kali)-[~]
└─# nmap -sV 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 07:43 EST
Nmap scan report for 192.168.29.163
Host is up (0.0017s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE  SERVICE     VERSION
21/tcp   open   ftp         ProFTPD 1.3.5
22/tcp   open   ssh         OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux;
 protocol 2.0)
80/tcp   open   http        Apache httpd 2.4.7
445/tcp  open   netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
631/tcp  open   ipp         CUPS 1.7
3000/tcp closed ppp
3306/tcp open   mysql       MySQL (unauthorized)
8080/tcp open   http        Jetty 8.1.7.v20120910
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Service Info: Hosts: 127.0.0.1, METASPLOITABLE3-UB1404; OSs: Unix, Linux; CPE
: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://n
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.28 seconds
```

Exception / When to use this: This scan is executed **only** after basic port discovery to enrich findings with actionable version data. Running `-sV` from the start increases scan time significantly. On Metasploitable, this reveals critical information like `vsftpd 2.3.4` (backdoor vulnerability), `OpenSSH 4.7p1`, and `Apache 2.2.8` — all known vulnerable versions that guide the exploitation phase.

Phase 2.11: OS Detection

```
nmap -O 192.168.29.163
```

Explanation: Uses TCP/IP stack fingerprinting to identify the operating system. Nmap sends a series of specially crafted packets and analyzes the response characteristics (TTL values, TCP window sizes, IP flags, IPID patterns) to match against its OS fingerprint database.

```
└─# nmap -O 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 07:45 EST
Nmap scan report for 192.168.29.163
Host is up (0.0018s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   open   ftp
22/tcp   open   ssh
80/tcp   open   http
445/tcp  open   microsoft-ds
631/tcp  open   ipp
3000/tcp closed ppp
3306/tcp open   mysql
8080/tcp open   http-proxy
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Aggressive OS guesses: Linux 3.2 - 4.14 (98%), Linux 3.8 - 3.16 (98%), Linux
3.10 - 4.11 (94%), Linux 3.13 - 4.4 (94%), Linux 3.13 (94%), Linux 3.13 - 3.1
6 (94%), OpenWrt Chaos Calmer 15.05 (Linux 3.18) or Designated Driver (Linux
4.1 or 4.4) (94%), Linux 4.10 (94%), Android 5.0 - 6.0.1 (Linux 3.4) (94%), A
ndroid 8 - 9 (Linux 3.18 - 4.4) (94%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.o
rg/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.81 seconds
```

Exception / When to use this: This scan is executed **only** when the OS is unknown and determining it is critical for selecting exploitation techniques. It requires at least one open and one closed TCP port to be effective. On Metasploitable, this correctly identifies the target as Linux 2.6.x, which guides exploit selection (e.g., Linux-specific kernel exploits, ELF binaries).

Phase 2.12: Aggressive Scan

```
nmap -A 192.168.29.163
```

Explanation: Enables a comprehensive scan package combining OS detection (`-O`), version detection (`-sV`), script scanning (`-sC`), and traceroute (`--traceroute`) in a single command. This is the most informative single-command scan, gathering maximum intelligence about the target.

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 07:46
EST
Nmap scan report for 192.168.29.163
Host is up (0.0021s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE  SERVICE     VERSION
21/tcp   open   ftp         ProFTPD 1.3.5
```

```
22/tcp   open   ssh         OpenSSH 6.6.1p1 Ubuntu 2ubuntu
2.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   1024 2b:2e:1f:a4:54:26:87:76:12:26:59:58:0d:da:3b:04 (D
SA)
|   2048 c9:ac:70:ef:f8:de:8b:a3:a3:44:ab:3d:32:0a:5c:6a (R
SA)
|   256 c0:49:cc:18:7b:27:a4:07:0d:2a:0d:bb:42:4c:36:17 (EC
DSA)
|_  256 a0:76:f3:76:f8:f0:70:4d:09:ca:e1:10:fd:a9:cc:0a (ED
25519)
80/tcp   open   http        Apache httpd 2.4.7
| http-ls: Volume /
| SIZE  TIME              FILENAME
| -     2020-10-29 19:37  chat/
| -     2011-07-27 20:17  drupal/
| 1.7K  2020-10-29 19:37  payroll_app.php
| -     2013-04-08 12:06  phpmyadmin/
|_
|_http-server-header: Apache/2.4.7 (Ubuntu)
|_http-title: Index of /
445/tcp  open   netbios-ssn Samba smbd 4.3.11-Ubuntu (workg
roup: WORKGROUP)
631/tcp  open   ipp         CUPS 1.7
|_http-server-header: CUPS/1.7 IPP/2.1
| http-robots.txt: 1 disallowed entry
|_/
| http-methods:
|_  Potentially risky methods: PUT
|_http-title: Home - CUPS 1.7.2
3000/tcp closed ppp
3306/tcp open   mysql       MySQL (unauthorized)
8080/tcp open   http        Jetty 8.1.7.v20120910
|_http-server-header: Jetty(8.1.7.v20120910)
|_http-title: Error 404 - Not Found
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle Vi
rtualBox virtual NIC)
```

```
Aggressive OS guesses: Linux 3.2 - 4.14 (98%), Linux 3.8 -
3.16 (98%), Linux 3.10 - 4.11 (94%), Linux 3.13 - 4.4 (9
4%), Linux 3.13 (94%), OpenWrt Chaos Calmer 15.05 (Linux 3.
18) or Designated Driver (Linux 4.1 or 4.4) (94%), Linux 4.
10 (94%), Android 5.0 - 6.0.1 (Linux 3.4) (94%), Android 8
- 9 (Linux 3.18 - 4.4) (94%), Linux 3.2 - 3.10 (94%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: Hosts: 127.0.0.1, METASPLOITABLE3-UB1404; OS
s: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled but not required
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
|   Computer name: metasploitable3-ub1404
|   NetBIOS computer name: METASPLOITABLE3-UB1404\x00
|   Domain name: \x00
|   FQDN: metasploitable3-ub1404
|_  System time: 2026-03-03T12:47:06+00:00
| smb2-time:
|   date: 2026-03-03T12:47:07
|_  start_date: N/A
|_clock-skew: mean: 0s, deviation: 2s, median: -1s

TRACEROUTE
HOP RTT     ADDRESS
1   2.10 ms 192.168.29.163

OS and Service detection performed. Please report any incor
rect results at https://nmap.org/submit/ .
```

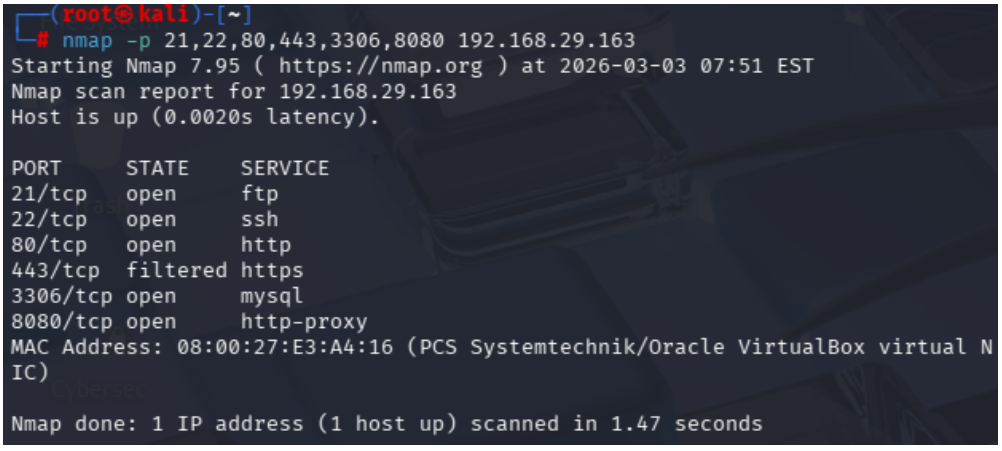
```
Nmap done: 1 IP address (1 host up) scanned in 57.51 second
s
```

Exception / When to use this: This scan is executed **only** when the assessment phase requires **maximum information gathering** in a single pass and **stealth is not a concern**. It is highly noisy and will trigger most IDS/IPS systems. On Metasploitable in a controlled lab environment, `-A` against 10.64.115.204 is the go-to command to produce a complete picture of all services, their versions, OS details, and network path — forming the foundation for the exploitation phase.

Phase 2.13: Specific Port Scan

```
nmap -p 21,22,80,443,3306,8080 192.168.29.163
```

Explanation: Restricts the scan to specified port numbers or ranges. This dramatically speeds up scanning by targeting only ports of interest. Port ranges can be specified as `1-1024`, individual ports as `21,22,80`, or combined.



Exception / When to use this: This is used **only** when the attacker already has intelligence about which services to target (e.g., from earlier reconnaissance) and a full 65535-port scan would be too time-consuming. On Metasploitable, this is useful for quickly re-confirming the status of known critical ports like FTP (21), SSH (22), HTTP (80), and MySQL (3306) without running a complete scan.

Phase 2.14: All Ports Scan

```
nmap -p- 192.168.29.163
```

Explanation: Scans all 65,535 TCP ports instead of nmap's default top 1,000 ports. This ensures no service running on a non-standard port is missed.

Exception / When to use this: This scan is executed **only** when thoroughness is required and there is suspicion that services may be running on non-standard or high ports. On Metasploitable, some services may run on unusual port numbers. While slow, `-p-` is essential in a comprehensive assessment to ensure 100% port coverage and avoid missing backdoors or misconfigurations on high ports.

Phase 2.15: Top Ports Scan

```
nmap --top-ports 100 10.64.115.204
```

Explanation: Scans only the top N most commonly open ports based on nmap's frequency database. This provides a quick overview of the most likely attack surface without the time cost of a full scan.

Exception / When to use this: This scan is executed **only** during **time-constrained assessments** or initial triage where a quick picture is needed before diving deeper. On Metasploitable, `--top-ports 100` quickly surfaces the most critical services (FTP, SSH, HTTP, SMB, etc.) in seconds, allowing the tester to prioritize which services to investigate further.

Phase 2.16: INIT + ACK Chunk Scan (SCTP)

```
nmap -sY -v 192.168.1.64
```

Explanation: Sends SCTP INIT+ACK chunks to target ports. SCTP (Stream Control Transmission Protocol) is an alternative to TCP used in telecom and signaling networks. If a port is open, the target responds with an INIT-ACK chunk; if closed, it responds with an ABORT chunk. This is the SCTP equivalent of a TCP SYN scan.

```
┌──(root㉿kali)-[~]
└─# nmap -p- 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 07:56 EST
Nmap scan report for 192.168.29.163
Host is up (0.0014s latency).
Not shown: 65524 filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   open   ftp
22/tcp   open   ssh
80/tcp   open   http
445/tcp  open   microsoft-ds
631/tcp  open   ipp
3000/tcp closed ppp
3306/tcp open   mysql
3500/tcp open   rtmp-port
6697/tcp open   ircs-u
8080/tcp open   http-proxy
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 134.16 seconds
```

Exception / When to use this: This scan is executed **only** when the target is suspected to be running **SCTP-based services** — commonly found in VoIP infrastructure, SS7 signaling, and telecom carrier equipment. On standard Metasploitable, SCTP services are not present, but this scan is used when assessing telecom-grade Linux systems or when nmap's service detection hints at SCTP listeners on unusual ports.

Phase 2.17: Cookie ECHO Scan (SCTP)

```
nmap -sZ -v 192.168.1.64
```

Explanation: Sends SCTP COOKIE ECHO chunks to target ports. This is a more stealthy SCTP scan — open ports silently drop the packet (no response) while closed ports respond with an ABORT chunk. Since it doesn't complete the SCTP handshake, it avoids full association establishment and is harder to log than a full SCTP connect.

```
┌──(root㉿kali)-[~]
└─# nmap -sZ -v 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 08:02 EST
Initiating ARP Ping Scan at 08:02
Scanning 192.168.29.163 [1 port]
Completed ARP Ping Scan at 08:02, 0.06s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 08:02
Completed Parallel DNS resolution of 1 host. at 08:02, 0.01s elapsed
Initiating SCTP COOKIE-ECHO Scan at 08:02
Scanning 192.168.29.163 [52 ports]
Completed SCTP COOKIE-ECHO Scan at 08:02, 2.99s elapsed (52 total ports)
Nmap scan report for 192.168.29.163
Host is up (0.0014s latency).
All 52 scanned ports on 192.168.29.163 are in ignored states.
Not shown: 52 open|filtered sctp ports (no-response)
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 3.17 seconds
           Raw packets sent: 105 (4.188KB) | Rcvd: 1 (28B)
```

Exception / When to use this: This scan is executed **only** against **SCTP-capable targets** where stealth is required and the `-sY` INIT scan may have triggered alerts. Like XMAS and NULL scans for TCP, this exploits SCTP protocol behavior to infer port states without completing a full handshake. Rarely used outside telecom/carrier-grade environments — on Metasploitable it serves as a protocol awareness exercise.

Lab 2-03 Perform OS Discovery using Nmap NSE Scripts

Scenario:

The TechSec Solutions security team has completed port enumeration on Metasploitable (10.64.115.204). The next phase requires deeper intelligence gathering using Nmap's Scripting Engine (NSE) to extract service-specific information, identify vulnerabilities, and enumerate key protocols running on the target.

Solution:

Phase 1: Understanding Nmap Scripting Engine (NSE)

NSE allows nmap to run Lua-based scripts against discovered services. Scripts are categorized into classes such as `auth`, `broadcast`, `brute`, `default`, `discovery`, `dos`, `exploit`, `external`, `fuzzer`, `intrusive`, `malware`, `safe`, `version`, and `vuln`. Running

appropriate scripts extracts far more actionable intelligence than basic scanning alone.

Phase 2: Default Script Scan

```
nmap -sC 192.168.29.163
```

Explanation: Runs the set of default NSE scripts against the target. These scripts perform common tasks like banner grabbing, HTTP title extraction, SSH host key retrieval, FTP anonymous login check, and SSL certificate enumeration. Equivalent to `--script=default`.

```
same as -A but with only based on the script
```

Exception / When to use this: This is executed during **standard enumeration** when a broad but safe set of additional data is needed without running aggressive or intrusive scripts. On Metasploitable, `-sC` immediately reveals anonymous FTP access, HTTP server details, and SSH host keys — all critical intelligence gathered safely.

Phase 2.1: SMB OS Discovery

```
nmap --script smb-os-discovery.nse 192.168.29.163
```

Explanation: Uses SMB protocol negotiation to extract OS version, computer name, domain, workgroup, and system time from the target. This script queries SMB's session setup response which often contains detailed system information.

```
┌──(root㉿kali)-[~]
└─# nmap --script smb-os-discovery.nse 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 11:21 EST
Nmap scan report for 192.168.29.163
Host is up (0.0021s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   open   ftp
22/tcp   open   ssh
80/tcp   open   http
445/tcp  open   microsoft-ds
631/tcp  open   ipp
3000/tcp closed ppp
3306/tcp open   mysql
8080/tcp open   http-proxy
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Host script results:
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
|   Computer name: metasploitable3-ub1404
|   NetBIOS computer name: METASPLOITABLE3-UB1404\x00
|   Domain name: \x00
|   FQDN: metasploitable3-ub1404
|_  System time: 2026-03-03T16:21:24+00:00
```

Exception / When to use this: This script is executed **only** when SMB (port 445 or 139) is open on the target and OS detection via TCP/IP fingerprinting (`-O`) is insufficient or inaccurate. On Metasploitable, Samba is running, and this script extracts the OS version directly from the SMB handshake — more accurate than TCP stack fingerprinting in this case.

Phase 2.2: FTP Anonymous Login Check

```
nmap --script ftp-anon 192.168.29.163 -p 21
```

Explanation: Tests whether the FTP service accepts anonymous authentication (username: `anonymous`, password: any email). If successful, it lists readable directories, revealing potential data exposure or a staging point for further attacks.

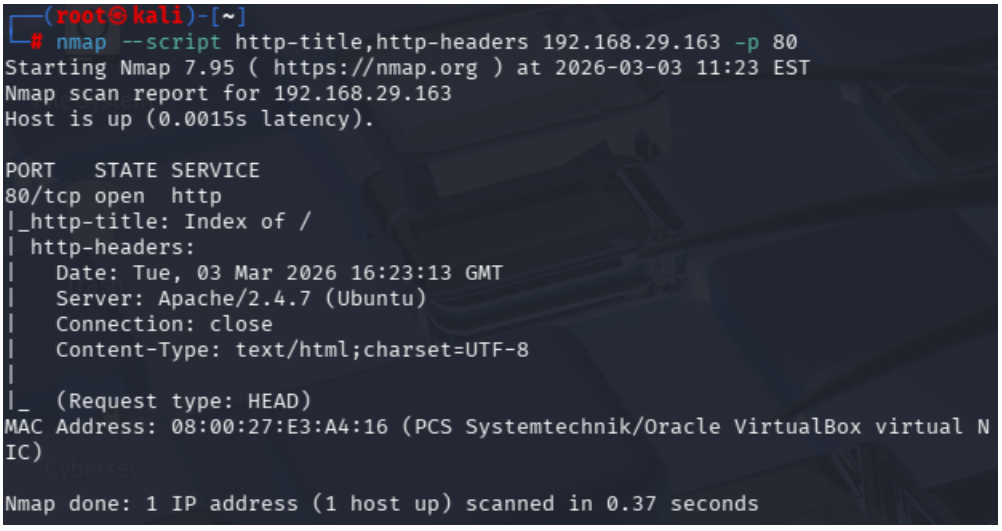
```
┌──(root㉿kali)-[~]
└─# nmap --script ftp-anon 192.168.29.163 -p 21
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 11:22 EST
Nmap scan report for 192.168.29.163
Host is up (0.00098s latency).

PORT   STATE SERVICE
21/tcp open  ftp
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 0.45 seconds
```

Exception / When to use this: This script is executed **only** when FTP (port 21) is discovered open. On Metasploitable, the vsftpd service allows anonymous login by default, making this a critical finding that directly indicates a misconfiguration allowing unauthenticated file access.

Phase 2.3: HTTP Title and Headers Enumeration

```
nmap --script http-title,http-headers 192.168.29.163 -p 80
```

Explanation: Retrieves the HTTP page title and server response headers from the web server. Headers reveal the server software, version, and sometimes framework details. The title provides application context.



```
┌──(root㉿kali)-[~]
└─# nmap --script http-title,http-headers 192.168.29.163 -p 80
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 11:23 EST
Nmap scan report for 192.168.29.163
Host is up (0.0015s latency).

PORT   STATE SERVICE
80/tcp open  http
|_http-title: Index of /
| http-headers:
|   Date: Tue, 03 Mar 2026 16:23:13 GMT
|   Server: Apache/2.4.7 (Ubuntu)
|   Connection: close
|   Content-Type: text/html;charset=UTF-8
|
|_  (Request type: HEAD)
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 0.37 seconds
```

Exception / When to use this: This script is run **only** when HTTP (port 80/443/8080) is open. On Metasploitable, this reveals Apache 2.2.8 running on Ubuntu, the PHP version, and the page title of the DVWA (Damn Vulnerable Web Application), giving context for web application attacks.

Phase 2.4: Vulnerability Script Scan

```
nmap --script vuln 192.168.29.163
```

Explanation: Runs all scripts in the `vuln` category against the target. These scripts test for known vulnerabilities including MS17-010 (EternalBlue), Heartbleed, ShellShock, and many others. This is a structured, non-exploit vulnerability check.

```
nmap --script vuln 192.168.29.163 -T5
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 11:29
EST
Nmap scan report for 192.168.29.163
Host is up (0.0011s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   open   ftp
22/tcp   open   ssh
80/tcp   open   http
| http-slowloris-check:
|   VULNERABLE:
|   Slowloris DOS attack
|     State: LIKELY VULNERABLE
|     IDs:  CVE:CVE-2007-6750
|       Slowloris tries to keep many connections to the tar
get web server open and hold
|       them open as long as possible.  It accomplishes thi
s by opening connections to
|       the target web server and sending a partial reques
t. By doing so, it starves
|       the http server's resources causing Denial Of Servi
ce.
|
|     Disclosure date: 2009-09-17
|     References:
|       http://ha.ckers.org/slowloris/
|_      https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-
2007-6750
|_http-stored-xss: Couldn't find any stored XSS vulnerabili
ties.
| http-csrf:
| Spidering limited to: maxdepth=3; maxpagecount=20; within
host=192.168.29.163
|   Found the following possible CSRF vulnerabilities:
|
|     Path: http://192.168.29.163:80/drupal/
|     Form id: user-login-form
```

```
|     Form action: /drupal/?q=node&destination=node
|
|     Path: http://192.168.29.163:80/payroll_app.php
|     Form id:
|     Form action:
|
|     Path: http://192.168.29.163:80/chat/
|     Form id: name
|_    Form action: index.php
| http-fileupload-exploiter:
|
|_    Couldn't find a file-type field.
| http-dombased-xss:
| Spidering limited to: maxdepth=3; maxpagecount=20; within
host=192.168.29.163
|   Found the following indications of potential DOM based
XSS:
|
|     Source: eval("document.location.href = '"+b+"pos="+a.
options[a.selectedIndex].value+"'")
|_    Pages: http://192.168.29.163:80/phpmyadmin/js/functio
ns.js?ts=1365422810
| http-enum:
|   /: Root directory w/ listing on 'apache/2.4.7 (ubuntu)'
|   /phpmyadmin/: phpMyAdmin
|_  /uploads/: Potentially interesting directory w/ listing
on 'apache/2.4.7 (ubuntu)'
| http-sql-injection:
|   Possible sqli for queries:
|     http://192.168.29.163:80/?C=S%3BO%3DA%27%20OR%20sqlsp
ider
|     http://192.168.29.163:80/?C=N%3BO%3DD%27%20OR%20sqlsp
ider
|     http://192.168.29.163:80/?C=M%3BO%3DA%27%20OR%20sqlsp
ider
|_    http://192.168.29.163:80/?C=D%3BO%3DA%27%20OR%20sqlsp
ider
445/tcp  open   microsoft-ds
```

```
631/tcp  open   ipp
|_http-aspnet-debug: ERROR: Script execution failed (use -d
to debug)
|_http-vuln-cve2014-3704: ERROR: Script execution failed (u
se -d to debug)
| http-enum:
|   /admin.php: Possible admin folder
|   /admin/: Possible admin folder
|   /admin/admin/: Possible admin folder
|   /administrator/: Possible admin folder
|   /adminarea/: Possible admin folder
|   /adminLogin/: Possible admin folder
|   /admin_area/: Possible admin folder
|   /administratorlogin/: Possible admin folder
|   /admin/account.php: Possible admin folder
|   /admin/index.php: Possible admin folder
|   /admin/login.php: Possible admin folder
|.........
3000/tcp closed ppp
3306/tcp open   mysql
8080/tcp open   http-proxy
| http-slowloris-check:
|   VULNERABLE:
|   Slowloris DOS attack
|     State: LIKELY VULNERABLE
|     IDs:  CVE:CVE-2007-6750
|       Slowloris tries to keep many connections to the tar
get web server open and hold
|       them open as long as possible.  It accomplishes thi
s by opening connections to
|       the target web server and sending a partial reques
t. By doing so, it starves
|       the http server's resources causing Denial Of Servi
ce.
|
|     Disclosure date: 2009-09-17
|     References:
|       http://ha.ckers.org/slowloris/
```

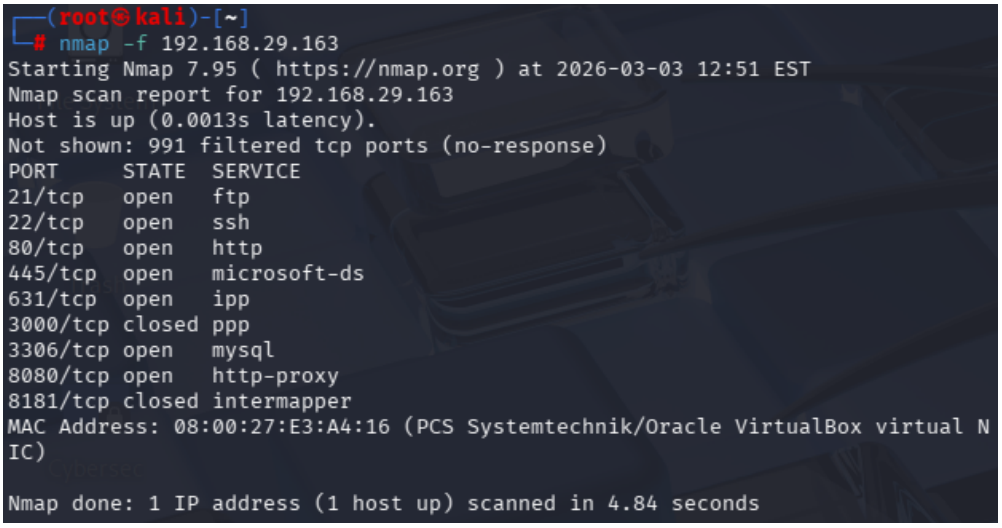
```
|_      https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-
2007-6750
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle Vi
rtualBox virtual NIC)

Host script results:
|_smb-vuln-ms10-054: false
|_smb-vuln-ms10-061: false
| smb-vuln-regsvc-dos:
|   VULNERABLE:
|   Service regsvc in Microsoft Windows systems vulnerable
to denial of service
|     State: VULNERABLE
|       The service regsvc in Microsoft Windows 2000 system
s is vulnerable to denial of service caused by a null defer
ence
|       pointer. This script will crash the service if it i
s vulnerable. This vulnerability was discovered by Ron Bowe
s
|       while working on smb-enum-sessions.
|_

Nmap done: 1 IP address (1 host up) scanned in 358.70 secon
ds

┌──(root㉿kali)-[~]
└─#
```

Exception / When to use this: This scan is executed **only** during the **vulnerability assessment phase** after port discovery, not during initial reconnaissance. It is intrusive and may crash unstable services. On Metasploitable, this produces a comprehensive list of known vulnerabilities including the vsftpd 2.3.4 backdoor, providing direct justification for exploitation attempts in subsequent lab phases.

Phase 2.5: Firewall / IDS Evasion — Fragmented Packets

```
nmap -f 192.168.29.163
```

Explanation: Fragments TCP packets into smaller 8-byte pieces. Some older firewalls and IDS systems cannot reassemble and inspect fragmented packets, allowing the scan to pass through undetected.



Exception / When to use this: This technique is used **only** when standard scans are being blocked or flagged by a packet inspection device. On Metasploitable in a controlled lab, this is used to demonstrate evasion principles — in production environments, modern Next-Generation Firewalls (NGFW) can reassemble and inspect fragmented packets, limiting the effectiveness of this technique.

Phase 2.6: Decoy Scan

```
nmap -D RND:10 192.168.29.163
```

Explanation: Mixes the attacker's real IP with 10 randomly generated decoy IPs in the scan packets. The target and any monitoring systems see traffic from multiple sources simultaneously, making it difficult to identify the true scanning origin.

```
┌──(root㉿kali)-[~]
└─# nmap -D RND:10 192.168.29.163
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 12:49 EST
Nmap scan report for 192.168.29.163
Host is up (0.0030s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT     STATE  SERVICE
21/tcp   open   ftp
22/tcp   open   ssh
80/tcp   open   http
445/tcp  open   microsoft-ds
631/tcp  open   ipp
3000/tcp closed ppp
3306/tcp open   mysql
8080/tcp open   http-proxy
8181/tcp closed intermapper
MAC Address: 08:00:27:E3:A4:16 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Nmap done: 1 IP address (1 host up) scanned in 10.55 seconds
```

Exception / When to use this: This technique is executed **only** when attribution evasion is required in the engagement. The decoy IPs must be live hosts on the network to avoid generating excessive traffic anomalies. On Metasploitable, `RND:10` generates random decoy IPs — useful for demonstrating how real-world attackers obscure their origin during reconnaissance.

Phase 2.7: Timing Templates

```
# Slowest - maximum stealth
nmap -T0 192.168.29.163

# Sneaky
nmap -T1 192.168.29.163

# Polite (reduces bandwidth)
nmap -T2 192.168.29.163

# Normal (default)
nmap -T3 192.168.29.163

# Aggressive (faster)
nmap -T4 192.168.29.163

# Insane (fastest, may miss results)
nmap -T5 192.168.29.163
```

Explanation: Timing templates control the speed and aggressiveness of the scan. T0 (Paranoid) introduces minutes between probes to evade time-based IDS correlation. T5 (Insane) sends probes as fast as the network allows, potentially dropping packets on congested links.

Exception / When to use this: `-T0` or `-T1` are used **only** when IDS/IPS evasion is critical and time is not a constraint. `-T4` is used in lab environments like Metasploitable where speed matters and no IDS is present. `-T5` is used **only** on local networks with high bandwidth and no packet loss. The default `-T3` is the safe choice for most assessments.

Phase 2.8: Output Formats

```
# Normal output
nmap 192.168.29.163 -oN scan_results.txt

# XML output (for import into tools like Metasploit)
nmap 192.168.29.163 -oX scan_results.xml

# Grepable output
nmap 192.168.29.163-oG scan_results.gnmap

# All formats simultaneously
nmap 192.168.29.163 -oA full_scan
```

Explanation: Nmap supports multiple output formats for different use cases. Normal output is human-readable. XML is machine-parseable and importable into frameworks. Grepable format allows quick `grep` / `awk` parsing. `-oA` saves all three simultaneously.

Exception / When to use this: `-oX` is used **only** when the results need to be imported into Metasploit (`db_import`), Faraday, or other vulnerability management platforms. `-oN` is used for documentation. On Metasploitable assessments, always saving with `-oA` ensures findings are preserved in all formats for reporting and toolchain integration.

Phase 2.9: Verbosity and Debugging

```
nmap -v 192.168.29.163  # Verbose
nmap -vv 192.168.29.163   # More verbose
nmap -d 192.168.29.163    # Debug level 1
nmap -d9 192.168.29.163   # Maximum debug
```

Explanation: Verbosity (`-v`) increases the amount of information displayed during scanning, showing open ports as they are discovered in real time. Debug mode (`-d`) outputs internal nmap logic including packet send/receive details for troubleshooting.

Exception / When to use this: `-v` is used **routinely** on all scans to see results as they arrive rather than waiting for completion. `-d` is used **only** when a scan produces unexpected or no results and the cause needs to be diagnosed — for example, when Metasploitable appears offline but is confirmed running, debug mode helps identify whether probes are being sent and if responses are being received.

Phase 2.10: Spoof MAC Scan

```
nmap --spoof-mac 0 192.168.29.163
# or use a specific vendor MAC
nmap --spoof-mac Cisco 192.168.29.163
# or a fully custom MAC
nmap --spoof-mac 00:11:22:33:44:55 192.168.29.163
```

Explanation: Spoofs the source MAC address in scan packets. Using `0` generates a random MAC each time. You can also specify a vendor name (e.g., `Cisco`, `Apple`) and nmap picks a real OUI from that vendor, or provide a fully custom MAC. This changes what the target and any network monitoring devices see as the scanner's hardware identity.

Exception / When to use this: This technique is executed **only** when MAC-based access controls (like 802.1X port authentication or MAC whitelisting) are in place on the network, or when evading MAC-level logging on managed switches. It **only works on the local subnet** — MAC addresses are stripped at Layer 3 boundaries (routers). On Metasploitable's local LAN, this prevents the target's ARP table and switch port logs from recording the true attacker MAC address.

Lab 2-04: Scan a Target Network using Metasploit

Scenario:

As a penetration tester, scanning a target network using Metasploit is a critical component of the overall assessment process. Metasploit enables the identification of open ports and active services across the network, serving as a foundational step for further enumeration and exploitation. Its extensive built-in module library allows testers to rapidly determine whether target systems are susceptible to known vulnerabilities, streamlining the path from discovery to exploitation.

Solution:

Phase 1: Document Scope and Understanding

Before initiating any Metasploit-based scanning, it is essential to establish the operational context. Metasploit integrates directly with a PostgreSQL database, allowing all discovered hosts, services, and vulnerabilities to be persistently stored and queried throughout the engagement. This makes it significantly more powerful than standalone scanning tools for large-scope assessments.

Lab Environment:

- **Attacker Machine:** Parrot OS / Kali Linux
- **Target Network:** 192.168.2.0/24
- **Target Host:** 192.168.2.100

Phase 2: Setting Up Metasploit with Database

Step 1 — Switch to root and initialize the database:

```
sudo su
msfdb init
```

`msfdb init` creates and configures the PostgreSQL database that Metasploit uses to store all scan results, hosts, services, and credentials gathered during the engagement. This must be done before launching `msfconsole` for persistent data storage.

Step 2 — Start PostgreSQL service:

```
service postgresql start
```

PostgreSQL is the backend database engine that Metasploit connects to. Starting it before launching `msfconsole` ensures the database connection is available immediately on startup.

Step 3 — Launch Metasploit Framework:

```
msfconsole
```

Step 4 — Verify database connectivity:

```
db_status
```

Expected output: `Connected to msf. Connection type: postgresql` — confirms Metasploit is properly linked to its database and all subsequent scan data will be saved.

Phase 3: Network Scanning via Nmap inside Metasploit

Step 5 — Run aggressive Nmap scan against entire subnet:

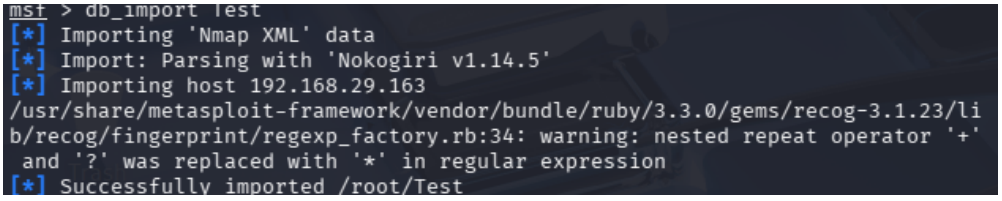
```
nmap -Pn -sS -A -oX Test 192.168.29.163
```

Flag Breakdown:

Flag	Meaning	Why Used
`-sS`	TCP SYN (stealth) scan	Half-open scan — faster and less likely to be logged than a full connect scan
`-oX Test`	XML output to file named "Test"	Saves results in XML format for direct import into Metasploit's database
`-Pn`	Skip host discovery (treat all hosts as up)	Used when ICMP ping is blocked by firewalls, ensuring no live host is skipped
`-A`	Aggressive mode	Enables OS detection, version detection, script scanning, and traceroute in one pass

Step 6 — Import scan results into Metasploit database:

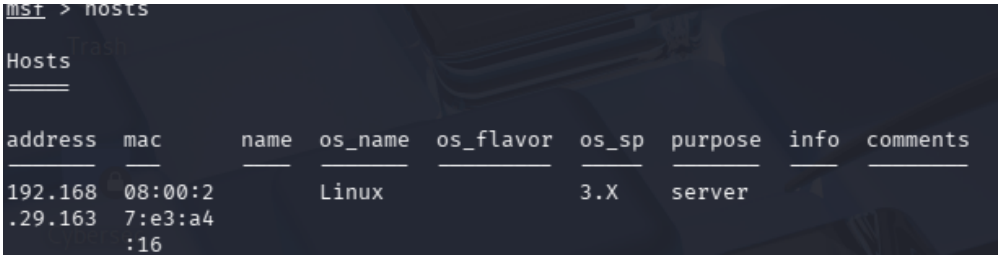
```
db_import Test
```



Parses the XML output from the Nmap scan and populates the Metasploit database with all discovered hosts and services. This enables querying and module targeting without re-scanning.

Step 7 — View all discovered hosts:

```
hosts
```



Lists all hosts discovered during the scan including their IP addresses, MAC addresses, OS details, and hostnames — pulled directly from the imported Nmap data.

Step 8 — View all open ports and services:

```
services
```

```
msf > services
Services
========

host            port  proto  name          state   info
----            ----  -----  ----          -----   ----
192.168.29.163  21    tcp    ftp           open    ProFTPD 1.3.5
192.168.29.163  22    tcp    ssh           open    OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 Ubuntu Linux; protocol 2.0
192.168.29.163  80    tcp    http          open    Apache httpd 2.4.7
192.168.29.163  445   tcp    netbios-ssn   open    Samba smbd 4.3.11-Ubuntu workgroup: WORKGROUP
192.168.29.163  631   tcp    ipp           open    CUPS 1.7
192.168.29.163  3000  tcp    ppp           closed
192.168.29.163  3306  tcp    mysql         open    MySQL unauthorized
192.168.29.163  8080  tcp    http          open    Jetty 8.1.7.v20120910
192.168.29.163  8181  tcp    intermapper   closed
```

Displays a structured table of all open ports and their associated services across every discovered host. This gives a complete attack surface overview in a single command.

Phase 4: Port Scanning using Metasploit Auxiliary Modules

Step 9 — Search for port scanning modules:

```
search portscan
```

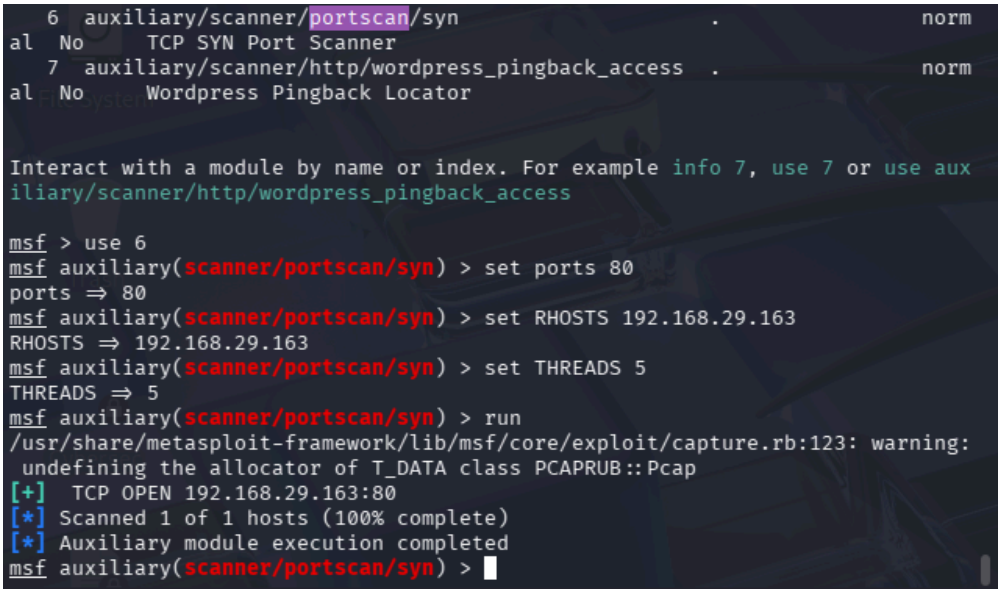
```
   #  Name                                                  Disclosure Date  Rank
    Check  Description
   -  ----  -----------                                     ---------------  ----
   0  auxiliary/scanner/portscan/ftpbounce                   .                norm
al  No     FTP Bounce Port Scanner
   1  auxiliary/scanner/natpmp/natpmp_portscan               .                norm
al  No     NAT-PMP External Port Scanner
   2  auxiliary/scanner/sap/sap_router_portscanner           .                norm
al  No     SAPRouter Port Scanner
   3  auxiliary/scanner/portscan/xmas                        .                norm
al  No     TCP "XMas" Port Scanner
   4  auxiliary/scanner/portscan/ack                         .                norm
al  No     TCP ACK Firewall Scanner
   5  auxiliary/scanner/portscan/tcp                         .                norm
al  No     TCP Port Scanner
   6  auxiliary/scanner/portscan/syn                         .                norm
al  No     TCP SYN Port Scanner
   7  auxiliary/scanner/http/wordpress_pingback_access       .                norm
al  No     Wordpress Pingback Locator
```

Lists all Metasploit auxiliary modules containing the keyword `portscan` — including SYN, TCP, ACK, FTP bounce, and XMAS scanners. This helps identify the right module for the specific scan type needed.

Step 10 — SYN Port Scan Module:

```
use auxiliary/scanner/portscan/syn
set PORTS 80
set RHOSTS 192.168.2.100
set THREADS 5
run
```

Explanation: Performs a TCP SYN scan against the target. Sends SYN packets and interprets SYN/ACK (open) or RST (closed) responses. The `THREADS` parameter controls parallel execution — 5 threads means 5 ports are probed simultaneously, significantly speeding up the scan.



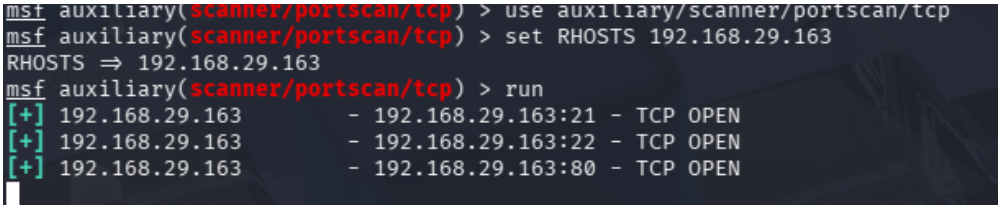
Exception / When to use this: This module is used **only** when a stealthy, non-completing TCP scan is required against a specific host or port range. It is preferred over the TCP module when IDS evasion is a concern since full connections are never established.

```
back
```

Step 11 — TCP Port Scan Module:

```
use auxiliary/scanner/portscan/tcp
set RHOSTS 192.168.29.163
run
```

Explanation: Performs full TCP connect scans against the target. Unlike the SYN module, this completes the three-way handshake, making it more reliable but also more visible in logs.



Exception / When to use this: This module is used **only** when raw socket access is unavailable (non-root context) or when reliability is prioritized over stealth — for example, scanning through certain proxies or when SYN responses are being silently dropped.

```
back
```

Step 12 — SMB Version Detection:

```
use auxiliary/scanner/smb/smb_version
set RHOSTS 192.168.29.163
set THREADS 10
run
```

Explanation: Probes SMB (ports 139/445) on the target to extract the exact SMB protocol version, OS name, computer name, domain, and workgroup. SMB version information is directly correlated to known critical vulnerabilities (e.g., EternalBlue targets SMBv1).

```
msf > use auxiliary/scanner/smb/smb_version
msf auxiliary(scanner/smb/smb_version) > set RHOSTS 192.168.29.163
RHOSTS => 192.168.29.163
msf auxiliary(scanner/smb/smb_version) > set THREADS 10
THREADS => 10
msf auxiliary(scanner/smb/smb_version) > run
[*] 192.168.29.163:445    - SMB Detected (versions:1, 2, 3) (preferred dialec
t:SMB 3.1.1) (compression capabilities:) (encryption capabilities:AES-128-CCM
) (signatures:optional) (guid:{00000000-0000-0000-0000-000000000000}) (authen
tication domain:METASPLOITABLE3-UB1404)
[+] 192.168.29.163:445    -   Host is running Version 6.1.0 (unknown OS)
[*] 192.168.29.163        - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/smb/smb_version) >
```

Exception / When to use this: This module is used **only** when SMB ports (139 or 445) are found open during port scanning. It is a mandatory step before attempting any SMB-based exploitation — identifying SMBv1 on the target immediately flags EternalBlue (MS17-010) as a potential attack vector.

Lab 2-05: Scan a Target using ShellGPT

Scenario:

The TechSec Solutions security team seeks to accelerate their network scanning workflow by integrating AI-assisted command generation. ShellGPT enables natural language instructions to be translated directly into precise shell commands, reducing manual effort and allowing testers to focus on analysis rather than syntax recall. This lab demonstrates how ShellGPT can orchestrate a complete network scanning workflow — from host discovery to vulnerability scanning — using conversational prompts.

Solution:

Phase 1: Document Scope and Understanding

ShellGPT is a GPT-powered command-line assistant that generates and executes shell commands from natural language input. In a penetration testing context, it functions as an intelligent command synthesizer — accepting high-level task descriptions and producing properly structured tool invocations. The `--chat` flag maintains conversation context across multiple commands, allowing progressive, stateful scanning workflows.

Lab Environment:

- **Attacker Machine:** Parrot OS
- **Target IP:** 10.0.2.20 / 10.0.2.15
- **Target Network:** 10.0.2.0/24

Prerequisites:

```
pip install shell-gpt
export OPENAI_API_KEY="your_api_key_here"
```

Phase 2: Host Discovery and Initial Scanning

Step 1 — ICMP Scanning using Hping3:

```
sgpt --chat scan --shell "Use Hping3 to perform ICMP scanni
ng on the target IP address 10.0.2.20 and stop after 10 ite
rations"
# At prompt: type E and press Enter
```

Hping3 sends crafted ICMP packets and measures round-trip time, confirming host liveness. The `10 iterations` limit ensures the scan terminates automatically — useful for a controlled, bounded test rather than continuous flooding.

Step 2 — ACK Scan using Hping3:

```
sgpt --chat scan --shell "Run a hping3 ACK scan on port 80
of target IP 10.0.2.20"
# Press Ctrl+C to stop
```

ACK scanning via Hping3 probes port 80 with TCP ACK packets to determine firewall filtering behavior — unfiltered ports return RST while filtered ports produce no response or ICMP errors.

Step 3 — Active Host Discovery across subnet:

```
sgpt --chat scan --shell "Scan the target network 10.0.2.0/
24 for active hosts and place only the IP addresses into a
file scan1.txt"
# At prompt: type E and press Enter
```

```
pluma scan1.txt   # view results
```

ShellGPT sweeps the full /24 subnet and extracts only live IPs into `scan1.txt` — the master target list for all subsequent scans in this lab.

Phase 3: Port and Service Scanning

Step 4 — Comprehensive Nmap scan against discovered hosts:

```
sgpt --chat scan --shell "Run a fast but comprehensive Nmap
scan against scan1.txt with low verbosity and write the res
ults to scan2.txt"

pluma scan2.txt   # view results
```

Runs a broad Nmap scan across all live hosts from `scan1.txt`, balancing speed and coverage, saving output to `scan2.txt` for documentation.

Step 5 — ICMP Echo Ping Sweep:

```
sgpt --chat scan --shell "Use Nmap to perform ICMP ECHO pin
g sweep on the target network 10.0.2.0/24"
# At prompt: type E and press Enter
```

A ping sweep using `-PE -sn` across the full subnet to enumerate all responding hosts — a quick baseline before running heavier port scans.

Step 6 — Open Port Discovery:

```
sgpt --chat scan --shell "Use Nmap to find open ports on ta
rget IP 10.0.2.20"
# At prompt: type E and press Enter
```

Runs a standard Nmap port scan against the specific target, displaying all open TCP ports and their associated service names.

Step 7 — Stealth Scan:

```
sgpt --chat scan --shell "Perform stealth scan on target IP
```

```
10.0.2.20 and display the results"
```

ShellGPT generates an `nmap -sS` command. The half-open technique avoids completing TCP handshakes, reducing the likelihood of detection by application-level logging.

Step 8 — XMAS Scan:

```
sgpt --chat scan --shell "Perform an XMAS scan on target IP
10.0.2.20"
```

Generates `nmap -sX` targeting 10.0.2.20. FIN+PSH+URG flags are set on probes — RFC-compliant Linux systems remain silent on open ports and respond with RST on closed ones.

Step 9 — Service Version Enumeration against all discovered hosts:

```
sgpt --chat scan --shell "Use Nmap to scan for open ports a
nd services against a list of IP addresses in scan1.txt and
copy only the port, service and version information with th
e respective IP address to a new file called scan3.txt"

pluma scan3.txt   # view results
```

Runs `-sV` version detection across all hosts in `scan1.txt`, extracting port/service/version into `scan3.txt` — a clean, actionable service inventory for vulnerability matching.

Phase 4: Advanced Scanning and Automation

Step 10 — Metasploit Port Discovery via ShellGPT:

```
sgpt --chat scan --shell "Use Metasploit to discover open p
orts on the IP address 10.0.2.15"
```

ShellGPT generates the appropriate `msfconsole` commands to run Metasploit's `auxiliary/scanner/portscan/tcp` module against 10.0.2.15 — demonstrating seamless AI-to-framework integration.

Step 11 — Full Service + MAC + Version Discovery:

```
sgpt --chat scan --shell "Use Nmap to scan open ports, MAC
details, services running on open ports with their versions
on target IP 10.0.2.20"
```

Generates a combined Nmap command (`-sS -sV -O --script=default`) retrieving open ports, MAC addresses, service banners, and version strings in a single pass.

Step 12 — OS Detection via TTL (10.0.2.20):

```
sgpt --chat scan --shell "Use TTL value and identify the op
erating system running on the target IP address 10.0.2.20,
display the TTL value and OS"
```

TTL-based OS fingerprinting: Linux returns TTL ≈ 64, Windows returns TTL ≈ 128, network devices return TTL ≈ 255 — a quick passive OS indicator without active fingerprinting.

Step 13 — OS Detection via TTL (10.0.2.15):

```
sgpt --chat scan --shell "Use TTL value and identify the op
erating system running on the target IP address 10.0.2.15,
display the TTL value and OS"
```

Same TTL-based methodology applied to the second target for comparative OS identification across both lab machines.

Step 14 — NSE OS Discovery against all hosts:

```
sgpt --chat scan --shell "Use Nmap script engine to perform
OS discovery on the target IP addresses in scan1.txt"
```

Generates an Nmap command using `--script smb-os-discovery` combined with `-O` across all hosts in `scan1.txt` — dual-method OS identification via TCP/IP fingerprinting and SMB negotiation.

Step 15 — Full Automation Script:

```
sgpt --chat scan --shell "Develop a script which will autom
ate network scanning efforts and find out live systems, ope
n ports, running services, service versions, etc. on target
```

```
IP range 10.0.2.0/24"
# At prompt: type E and press Enter
```

ShellGPT generates a comprehensive bash script chaining host discovery, port scanning, and service enumeration into one automated workflow across the entire /24 range.

Step 16 — IP Address Decoy Evasion:

```
sgpt --chat scan --shell "To evade an IDS/Firewall, use IP
address decoy technique to scan the target IP address 10.0.
2.15"
```

Generates `nmap -D RND:10` mixing the attacker's real IP with 10 random decoy IPs — making it significantly harder for IDS/IPS logs to identify the true scanning source.

Step 17 — Modify scan1.txt (Remove specific IPs):

```
sgpt --chat scan --shell "Within scan1.txt file remove 10.
0.2.3 and 10.0.2.19 entries, then display results"
```

Demonstrates ShellGPT's file manipulation capability — generating `sed` or `grep -v` commands to cleanly remove specific entries from the target list before proceeding.

Phase 5: Python Automation with Nmap + Nikto

Step 18 — Generate Python scanning script:

```
sgpt --chat scancode --code "Create a python script to run
a fast but comprehensive Nmap scan on the IP addresses in s
can1.txt and then execute vulnerability scanning using nikt
o against each IP address in scan1.txt"
```

ShellGPT generates a complete Python script iterating through `scan1.txt`, running `python-nmap` against each host, then sequentially executing `nikto` web vulnerability scans — a fully automated recon-to-vuln-scan pipeline.

Step 19 — Save the generated script:

```
pluma python_scan.py
```

```
# Paste the generated code, save and close
```

Step 20 — Execute the script:

```
python3 python_scan.py
```

Runs the AI-generated automation script, producing combined Nmap + Nikto output for all hosts in `scan1.txt`.

> **Note:** Always explore additional ShellGPT prompts beyond these steps. The `--chat` flag maintains session context, allowing progressive refinement through natural language follow-up instructions.

> **Disclaimer : Using the notes is for educational purpose only, the owner is not responsible for any misconduct by the reader**